



**RV College of
Engineering®**

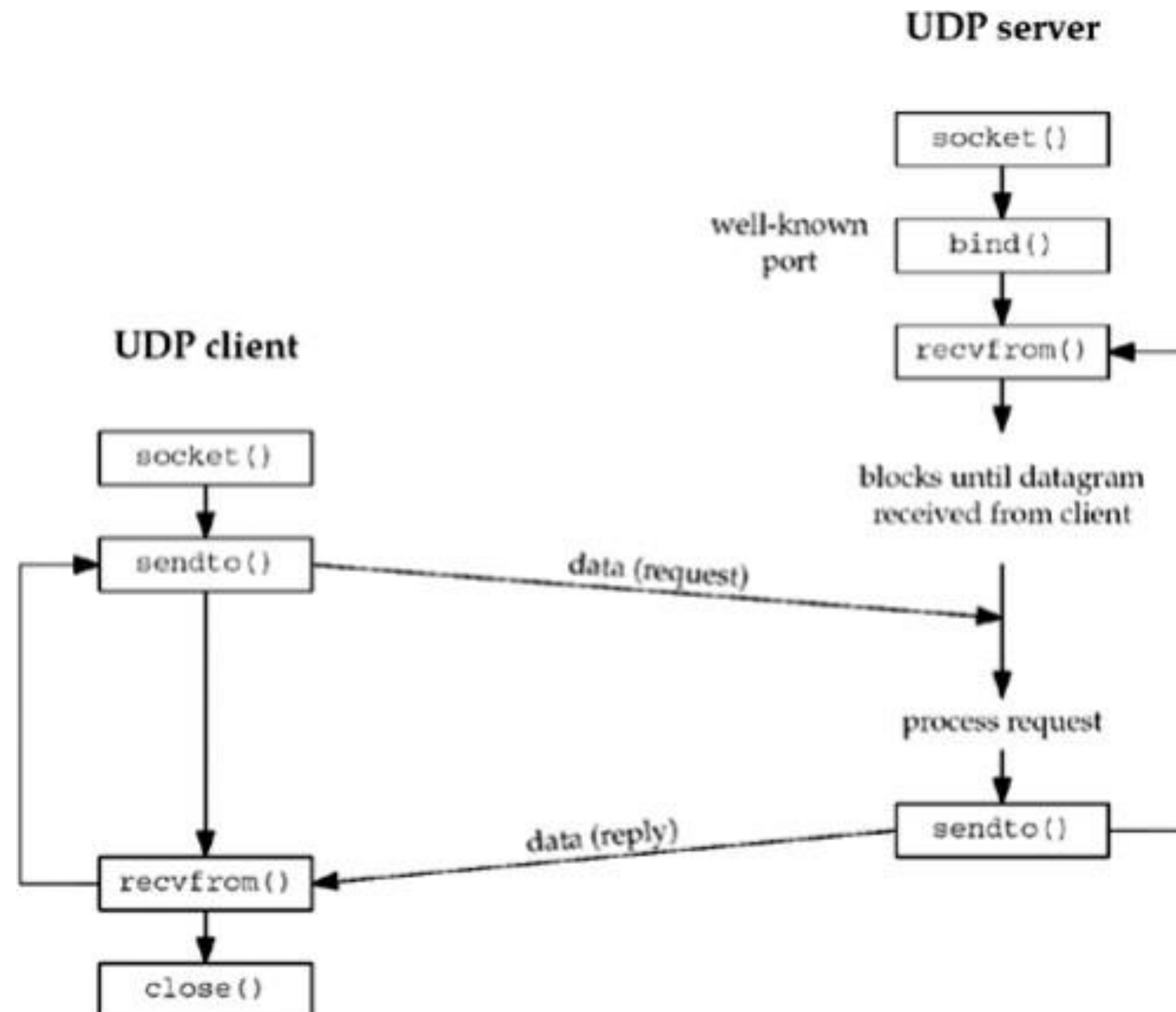
Go, change the world

Unit 3- UDP client/server

Introduction

- There are fundamental differences between applications written using TCP versus those that use UDP.
- It is because differences in UDP and TCP protocols.
- UDP is a connectionless, unreliable, datagram protocol.
- TCP is connection-oriented, reliable byte stream.
- Applications built using UDP are: DNS, NFS, and SNMP

Socket functions for UDP client and server



- The client does not establish a connection with the server.
- Instead, the client just sends a datagram to the server using the `sendto()`.
- `Sendto()` takes the address of the destination (the server) as a parameter.
- The server does not accept a connection from a client.
- Instead, the server calls the `recvfrom()`, which waits until data arrives from some client.
- `Recvfrom` returns the protocol address of the client, along with the datagram, so the server can send a response to the correct client.

sendto() and recvfrom()

```
#include <sys/socket.h>
```

```
ssize_t recvfrom(int sockfd, void *buff, size_t nbytes, int flags, struct sockaddr *  
from, socklen_t *addrlen);
```

```
ssize_t sendto(int sockfd, const void *buff, size_t nbytes, int flags, const struct  
sockaddr *to, socklen_t addrlen);
```

Both return: number of bytes read or written if OK, 1 on error

Function	Parameter	Type	Description	Key Insight
<code>sendto()</code>	<code>sockfd</code>	<code>int</code>	Socket descriptor used to send data	Identifies <i>which socket</i> is sending
<code>sendto()</code>	<code>buff</code>	<code>const void *</code>	Data to be sent	Source of outgoing message
<code>sendto()</code>	<code>nbytes</code>	<code>size_t</code>	Number of bytes to send	Usually <code>strlen(buff)</code>
<code>sendto()</code>	<code>flags</code>	<code>int</code>	Controls sending behavior (usually 0)	Rarely used in basic UDP
<code>sendto()</code>	<code>to</code>	<code>const struct sockaddr *</code>	Destination address (IP + port)	Specifies <i>where data goes</i>
<code>sendto()</code>	<code>addrlen</code>	<code>socklen_t</code>	Size of destination address	Typically <code>sizeof(struct sockaddr_in)</code>

Function	Parameter	Type	Description	Key Insight
recvfrom()	sockfd	int	Socket descriptor used to receive data	Identifies <i>which socket</i> is receiving
recvfrom()	buff	void *	Buffer to store incoming data	Destination memory for received message
recvfrom()	nbytes	size_t	Maximum number of bytes to read	Prevents buffer overflow
recvfrom()	flags	int	Controls receive behavior (usually 0)	Can enable non-blocking or special modes
recvfrom()	from	struct sockaddr *	Stores sender's address (IP + port)	Helps identify <i>who sent the data</i>
recvfrom()	addrlen	socklen_t *	Size of address structure (input/output)	Must be initialized before call

- The first three arguments, sockfd, buff, and nbytes, are identical to the first three arguments of send() and recv() : descriptor, pointer to buffer to read into or write from, and number of bytes to read or write.
- The flag argument is set to 0.
- to argument for sendto is a socket address structure containing the protocol address (e.g., IP address and port number) of where the data is to be sent.
- The size of this socket address structure is specified by addrlen.

- The recvfrom function fills in the socket address structure pointed to by **from** with the protocol address of who sent the datagram.
- The number of bytes stored in this socket address structure is also returned to the caller in the integer pointed to by addrlen.
- *Note:* the final argument to sendto is an integer value, while the final argument to recvfrom is a pointer to an integer value (a value-result argument).

- The **final two arguments** to `recvfrom` are similar to the final two arguments **to accept**: The contents of the socket address structure upon return tell us *who sent the datagram (in the case of UDP) or who initiated the connection (in the case of TCP)*.
- The final two arguments to `sendto` are similar to the final two arguments to **connect**: We fill in the socket address structure with the protocol address of *where to send the datagram (in the case of UDP) or with whom to establish a connection (in the case of TCP)*.

- Both functions return the length of the data that was read or written as the value of the function.
- In the typical use of `recvfrom`, with a datagram protocol, the return value is the amount of user data in the datagram received.
- Writing a datagram of length 0 is acceptable.
- In UDP, this results in an IP datagram containing an IP header (normally 20 bytes for IPv4 and 40 bytes for IPv6), an 8-byte UDP header, and no data.
- This also means that a return value of 0 from `recvfrom` is acceptable for a datagram protocol.
- It does not mean that the peer has closed the connection, as does a return value of 0 from `read` on a TCP socket. Since UDP is connectionless, there is no such thing as closing a UDP connection.

- If the from argument to recvfrom is a null pointer, then the corresponding length argument (addrlen) must also be a null pointer, and this indicates that we are not interested in knowing the protocol address of who sent us data.

Lost Datagrams

- UDP client/server example is not reliable.
- If a client datagram is lost (say it is discarded by some router between the client and server), the client will block forever in its call to `recvfrom`.
- Similarly, if the client datagram arrives at the server but the server's reply is lost, the client will again block forever in its call to `recvfrom`.
- A typical way to prevent this is to place a timeout on the client's call to `recvfrom`

Lost Datagrams

There are three ways to place a timeout on an I/O operation involving a socket:

- Call alarm, which generates the SIGALRM signal when the specified time has expired.
- Block waiting for I/O in select, which has a time limit built-in, instead of blocking in a call to read or write.
- Use the newer SO_RCVTIMEO and SO_SNDTIMEO socket options.

Domain Name Systems(DNS).

- The DNS is used primarily to map between hostnames and IP addresses.
- A hostname can be either a simple name, such as solaris or freebsd.
- A fully qualified domain name (FQDN) such as solaris.unpbook.com.
- An FQDN is also called an absolute name and must end with a DOT, but users often omit the ending period.
- The trailing DOT tells the resolver that this name is fully qualified and it doesn't need to search its list of possible domains.

- Entries in the DNS are known as resource records (RRs).
- Few types of RRs that are of interest are:
- A- An A record maps a hostname into a 32-bit IPv4 address.
- AAAA- A AAAA record, called a "quad A" record, maps a hostname into a 128-bit IPv6 address.
 - The term "quad A" is used because a 128-bit address is four times larger than a 32-bit address.
- PTR - PTR records (called "pointer records") map IP addresses into hostnames. PTR records are used for the reverse DNS lookup

- A reverse DNS lookup is the opposite of this process: it is a query that starts with the IP address and looks up the domain name.
- While DNS A records are stored under the given domain name, DNS PTR records are stored under the IP address — reversed, and with ".in-addr.arpa" added.
- For example, the PTR record for the IP address 192.0.2.255 would be stored under "255.2.0.192.in-addr.arpa".
- "in-addr.arpa" has to be added because PTR records are stored within the .arpa top-level domain in the DNS.
- .arpa is a domain used mostly for managing network infrastructure, and it was the first top-level domain name defined for the Internet.

- IPv6 addresses are constructed differently from IPv4 addresses.
- IPv6 PTR records exist in a different namespace within .arpa.
- IPv6 PTR records are stored under the IPv6 address, reversed and converted into four-bit sections (as opposed to 8-bit sections, as in IPv4), plus ".ip6.arpa".
- Uses for reverse DNS include:
 - Anti-spam: Some email anti-spam filters use reverse DNS to check the domain names of email addresses and see if the associated IP addresses are likely to be used by legitimate email servers.
 - Troubleshooting email delivery issues: Because anti-spam filters perform these checks, email delivery problems can result from a misconfigured or missing PTR record.
 - Logging: System logs typically record only IP addresses; a reverse DNS lookup can convert these into domain names for logs that are more human-readable.

- MX - An MX record specifies a host to act as a "mail exchanger" for the specified host.
- The MX record indicates how email messages should be routed in accordance with the Simple Mail Transfer Protocol

Example of an MX record:

example.com	record type:	priority:	value:	TTL
@	MX	10	mailhost1.example.com	45000
@	MX	20	mailhost2.example.com	45000

- The 'priority' numbers before the domains for these MX records indicate preference; the lower 'priority' value is preferred.
- The server will always try mailhost1 first because 10 is lower than 20. In the result of a message send failure, the server will default to mailhost2.

What is the process of querying an MX record?

- Message transfer agent (MTA) software is responsible for querying MX records.
- When a user sends an email, the MTA sends a DNS query to identify the mail servers for the email recipients.
- The MTA establishes an SMTP connection with those mail servers, starting with the prioritized domains (in the first example above, mailhost1).

- CNAME The ‘canonical name’ (CNAME) record is used in lieu of an A record, when a domain or subdomain is an alias of another domain.
- All CNAME records must point to a domain, never to an IP address.
- A common use is to assign CNAME records for common services, such as ftp and [www](#). If people use these service names instead of the actual hostnames, it is transparent when a service is moved to another host.

- For example, suppose blog.example.com has a CNAME record with a value of ‘example.com’ (without the ‘blog’).
- This means when a DNS server hits the DNS records for blog.example.com, it actually triggers another DNS lookup to example.com, returning example.com’s IP address via its A record.
- In this case we would say that example.com is the canonical name (or true name) of blog.example.com.
- Oftentimes, when sites have subdomains such as blog.example.com or shop.example.com, those subdomains will have CNAME records that point to a root domain (example.com).
- This way if the IP address of the host changes, only the DNS A record for the root domain needs to be updated and all the CNAME records will follow along with whatever changes are made to the root.



**RV College of
Engineering®**

Go, change the world

Socket Options

Topics: Introduction, *getsockopt* and *setsockopt*

Introduction

- In addition to binding a socket to a local address or connecting it to a destination address, application programs need a method to control the socket.
- For example,
 - When using protocols that use time-out and retransmission, the application program may want to obtain or set the time-out parameters.
 - An application program may also want to control the allocation of buffer space.
 - Determine if the socket allows transmission of broadcast, or control processing of out-of-band data

- The **getsockopt** subroutine allows an application program to request information about socket options.
- The **setsockopt** subroutine allows an application program to set a socket option using the same set of values obtained with the **getsockopt** subroutine.
- ❖ Not all socket options apply to all sockets.
- ❖ The options that can be set depend on the current state of the socket and the underlying protocol being used.

- sockfd must refer to an open socket descriptor.
- level specifies the code in the system that interprets the option: the general socket code or some protocol-specific code (e.g., IPv4, IPv6, TCP, or SCTP).
- optval is a pointer to a variable from which the new value of the option is fetched by getsockopt, or into which the current value of the option is stored by setsockopt.
- The size of this variable is specified by the final argument, as a value for setsockopt and as a value-result for getsockopt.

These two functions apply only to sockets.

```
#include <sys/socket.h>
```

```
int getsockopt(int sockfd, int level, int optname, void *optval, socklen_t *optlen);
```

```
int setsockopt(int sockfd, int level, int optname, const void *optval socklen_t  
optlen);
```

Both return: 0 if OK, 1 on error

setsockopt - set the socket options: Used to configure socket behavior (timeouts, reuse address, buffer size, etc.)

```
#include <sys/socket.h>
```

```
int setsockopt(int socket, int level, int option_name, const void *option_value,  
socklen_t option_len);
```

The setsockopt() function shall **set the option specified by the option_name** argument, at the **protocol level specified by the level argument**, to **the value pointed to by the option_value** argument for the socket associated with the file descriptor specified by the socket argument.

socket	Socket descriptor on which option is applied
level	Protocol level (e.g., SOL_SOCKET, IPPROTO_TCP)
option_name	Specific option to set (e.g., SO_REUSEADDR, SO_RCVTIMEO)
option_value	Pointer to value you want to assign to the option
option_len	Size of the value pointed by option_value

Example :

```
struct timeval tv;  
tv.tv_sec = 5;  
tv.tv_usec = 0;
```

```
setsockopt(sockfd,  
SOL_SOCKET, SO_RCVTIMEO,  
&tv, sizeof(tv));
```

setsockopt - set the socket options

- The *level* argument specifies the protocol level at which the option resides.
- To set *options* at the socket level, specify the level argument as SOL_SOCKET.
- To set options at other levels, supply the appropriate level identifier for the protocol controlling the option.
- For example, to indicate that an option is interpreted by the TCP (Transport Control Protocol), set level to IPPROTO_TCP as defined in the <netinet/in.h> header.

setsockopt - set the socket options

Some Options at socket level: SOL_SOCKET

1. **SO_REUSEADDR**: Specifies that the rules used in validating addresses supplied to `bind()` should allow reuse of local addresses, if this is supported by the protocol. This option takes an int value. This is a Boolean option.
2. **SO_KEEPALIVE** Keeps connections active by enabling the periodic transmission of messages, if this is supported by the protocol. This option takes an int value.
3. **SO_SNDBUF** -Sets send buffer size. This option takes an int value.
4. **SO_RCVBUF** Sets receive buffer size. This option takes an int value.

setsockopt - set the socket options

Some Options at IPLevel: IPPROTO_IP

1. IP_TTL
2. IP_MULTICAST_IF
3. IP_{ADD,DROP}_MEMBERSHIP

Some Options at TCPLevel: IPPROTO_TCP

1. TCP_MAXSEG
2. TCP_NODELAY

setsockopt - set the socket options

Upon successful completion, *setsockopt()* shall return 0. Otherwise, -1 shall be returned and *errno* set to indicate the error.

APPLICATION USAGE: The *setsockopt()* function provides an application program with the means to control socket behavior.

An application program can use *setsockopt()* to allocate buffer space, control timeouts, or permit socket data broadcasts.

getsockopt - get the socket options

```
#include <sys/socket.h>
```

```
int getsockopt(int socket, int level, int option_name, void * option_value,  
socklen_t *option_len);
```

- The getsockopt() function shall retrieve the **value for the option specified by the option_name** argument for the socket specified by the socket argument.
- If the size of the option value is greater than option_len, the value stored in the object pointed to by the option_value argument shall be silently truncated.
Otherwise, the object pointed to by the option_len argument shall be modified to indicate the actual length of the value.

getsockopt - get the socket options

- The level argument specifies the protocol level at which the option resides.
- To retrieve options at the socket level, specify the level argument as SOL_SOCKET.
- To retrieve options at other levels, supply the appropriate level identifier for the protocol controlling the option.
- For example, to indicate that an option is interpreted by the TCP (Transmission Control Protocol), set level to IPPROTO_TCP as defined in the <netinet/in.h> header.
- ❖ The socket in use may require the process to have appropriate privileges to use the getsockopt() function.

getsockopt - get the socket options

The option_name argument specifies a single option to be retrieved. It can be one of the following values defined in <sys/socket.h>:

- **SO_DEBUG**- Reports whether debugging information is being recorded. This option shall store an int value. This is a Boolean option.
- **SO_ACCEPTCONN** - Reports whether socket listening is enabled. This option shall store an int value. This is a Boolean option.
- **SO_BROADCAST** - Reports whether transmission of broadcast messages is supported, if this is supported by the protocol. This option shall store an int value. This is a Boolean option.
- **SO_REUSEADDR** - Reports whether the rules used in validating addresses supplied to bind() should allow reuse of local addresses, if this is supported by the protocol. This option shall store an int value. This is a Boolean option.

- Some of the socket and IP-layer and transport layer socket options for getsockopt and setsockopt.

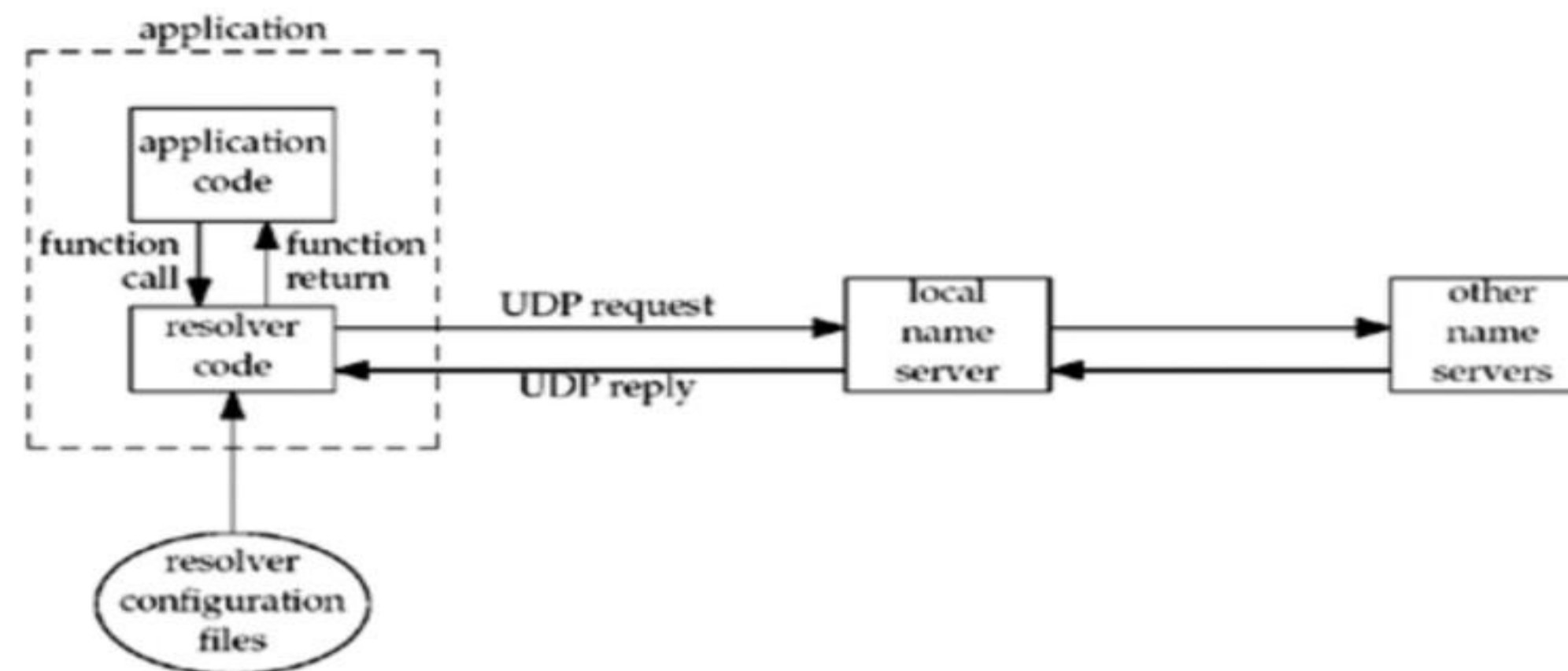
level	optname	get	set	Description	Flag	Datatype
IPPROTO_TCP	TCP_MAXSEG	•	•	Maximum segment size	•	int
IPPROTO_SCTCP	SCTCP_MAXSEG	•	•	Max fragmentation size		int
IPPROTO_IP or IPPROTO_IPV6	MCAST_JOIN_GROUP		•	Join multicast group		group_req{}
IPPROTO_IP	IP_TTL	•	•	TTL		int
SOL_SOCKET	SO_SNDBUF	•	•	Send buffer size		int

Aspect	<code>getsockopt()</code>	<code>setsockopt()</code>
Purpose	Retrieve (read) socket option value	Set (modify) socket option value
Direction of Data	From kernel → user	From user → kernel
<code>optval</code> role	Output parameter (stores result)	Input parameter (provides value)
<code>optlen</code> role	Input + Output (size given & updated)	Input only (size of value passed)
Usage	Check current configuration	Configure socket behavior
Example Meaning	“What is current timeout?”	“Set timeout to 5 seconds”
Data Flow Analogy	Reading a setting	Writing a setting
Return Value	0 → success, -1 → error	0 → success, -1 → error

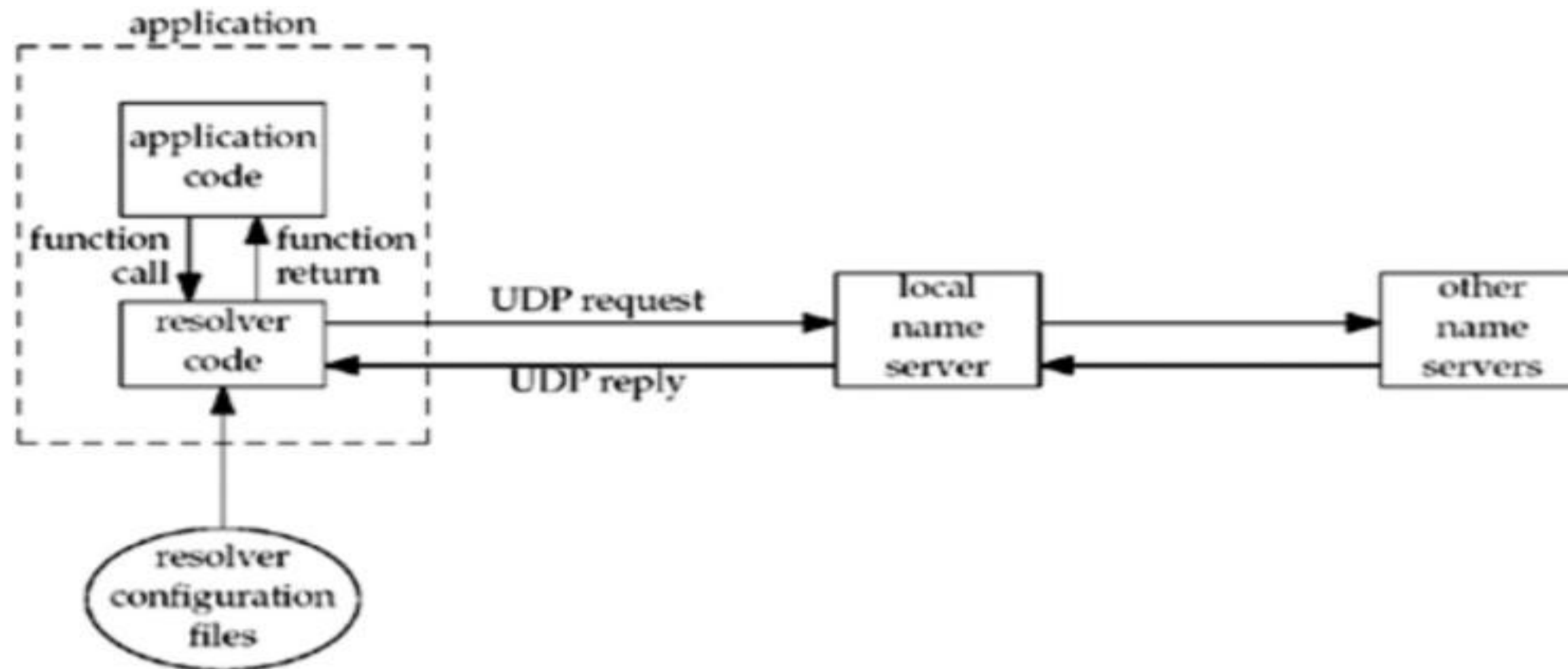
- There are two basic types of options:
 - binary options that enable or disable a certain feature (flags), and
 - options that fetch and return specific values that we can either set or examine (values).
- The column labelled "Flag" specifies if the option is a flag option.
- When calling getsockopt for these flag options, *optval is an integer.
- The value returned in *optval is zero if the option is disabled, or nonzero if the option is enabled. Similarly, setsockopt requires a nonzero *optval to turn the option on, and a zero value to turn the option off.
- If the "Flag" column does not contain a "•," then the option is used to pass a value of the specified datatype between the user process and the system.

Resolvers and Name Servers.

- Organizations run one or more name servers, often the program known as BIND (Berkeley Internet Name Domain).
- Applications such as the clients and servers that we are writing in this text contact a DNS server by calling functions in a library known as the resolver.
- The common resolver functions are `gethostbyname` and `gethostbyaddr`.
- The former maps a hostname into its IPv4 addresses, and the latter does the reverse mapping.



Arrangements of clients ,resolvers and name Servers.



- On some systems, the resolver code is contained in a system library and is link-edited into the application when the application is built.
- On others, there is a centralized resolver daemon that all applications share, and the system library code performs RPCs to this daemon.
- In either case, application code calls the resolver code using normal function calls, typically calling the functions `gethostbyname` and `gethostbyaddr`.
- The resolver code reads its system-dependent configuration files to determine the location of the organization's name servers.
- The file `/etc/resolv.conf` normally contains the IP addresses of the local name servers.

- The resolver sends the query to the local name server using UDP.
- If the local name server does not know the answer, it will normally query other name servers across the Internet, also using UDP.
- If the answers are too large to fit in a UDP packet, the resolver will automatically switch to TCP.
- Note: TCP can reliably transfer large amounts of data without size limitations and packet loss issues.

DNS Alternatives

- It is possible to obtain name and address information without using the DNS.
- Static host files (normally the file /etc/hosts), the Network Information System (NIS) or the Lightweight Directory Access Protocol (LDAP).
- It is implementation-dependent how an administrator configures a host to use the different types of name services.
- If a name server is to be used for hostname lookups, then all these systems use the file /etc/resolv.conf to specify the IP addresses of the name servers.
- Call the resolver functions such as gethostbyname and gethostbyaddr.

Gethostbyname Function

- Host computers are normally known by human-readable names.
- But functions, connect, accept, bind..... these use IP addresses.
- Most applications should deal with names, not addresses. (especially IPv6).
- The most basic function that looks up a hostname is gethostbyname.
- If successful, it returns a pointer to a hostent structure that contains all the IPv4 addresses for the host.
- However, it is limited in that it can only return IPv4 addresses.(newer variant is getaddrinfo())

Gethostbyname Function

```
#include <netdb.h>
```

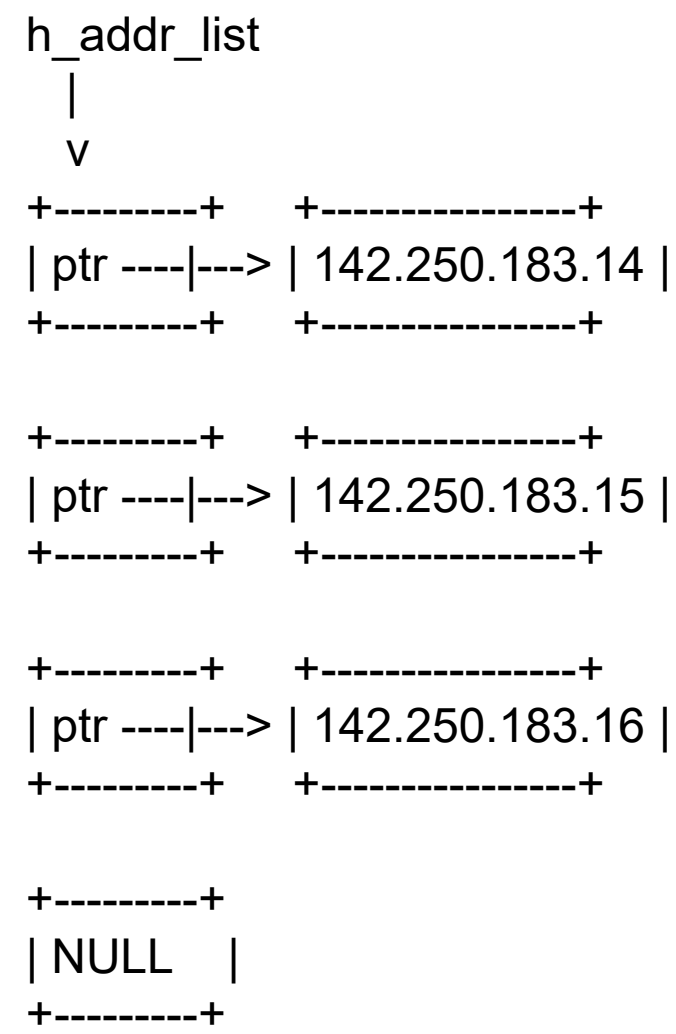
```
struct hostent *gethostbyname (const char *hostname);
```

Returns: non-null pointer if OK, `NULL` on error with `h_errno` set

The non-null pointer returned by this function points to the following `hostent` structure:

```
struct hostent {  
    char *h_name;           /* official (canonical) name of host */  
    char **h_aliases;       /* pointer to array of pointers to alias names */  
    int h_addrtype;         /* host address type: AF_INET */  
    int h_length;           /* length of address: 4 */  
    char **h_addr_list;     /* ptr to array of ptrs with IPv4 addrs */  
};
```

gethostbyname() - DNS Resolution Function								
Aspect	Explanation	Technical Insight	Example	Field	Type	Meaning	Why It Matters	Example Value
Purpose	Converts hostname → IP address	Performs DNS resolution using system resolver	"www.google.com" → 142.250.x.x	h_name	char *	Official hostname	Canonical DNS name	"www.google.com"
Header File	<netdb.h>	Contains networking database structures	Required in socket programming	h_aliases	char **	Alternate names	Rarely used, but shows DNS aliases	["google.com"]
Input	Hostname (string)	Domain name to resolve	"example.com"					
Return Type	struct hostent *	Pointer to DNS result structure	Non-NULL if success	h_addrtype	int	Address type	Identifies protocol	AF_INET
Failure Case	Returns NULL	Use h_errno for error details	Host not found					
Protocol Support	IPv4 only	Uses AF_INET	No IPv6 support ❌	h_length	int	Length of IP	Helps interpret address correctly	4 (IPv4 = 4 bytes)
Thread Safety	Not thread-safe	Uses static memory internally	Avoid in multithreading					
Usage Stage	Before connect()	Needed to get server IP	Client-side networking	h_addr_list ⭐	char **	List of IP addresses	Most important field	Multiple IPs
Modern Alternative	getaddrinfo()	Supports IPv4 + IPv6	Recommended ✔️					



Reason

Explanation

Load balancing

Traffic distributed across servers

Redundancy

Backup servers

Multiple network interfaces

Different networks

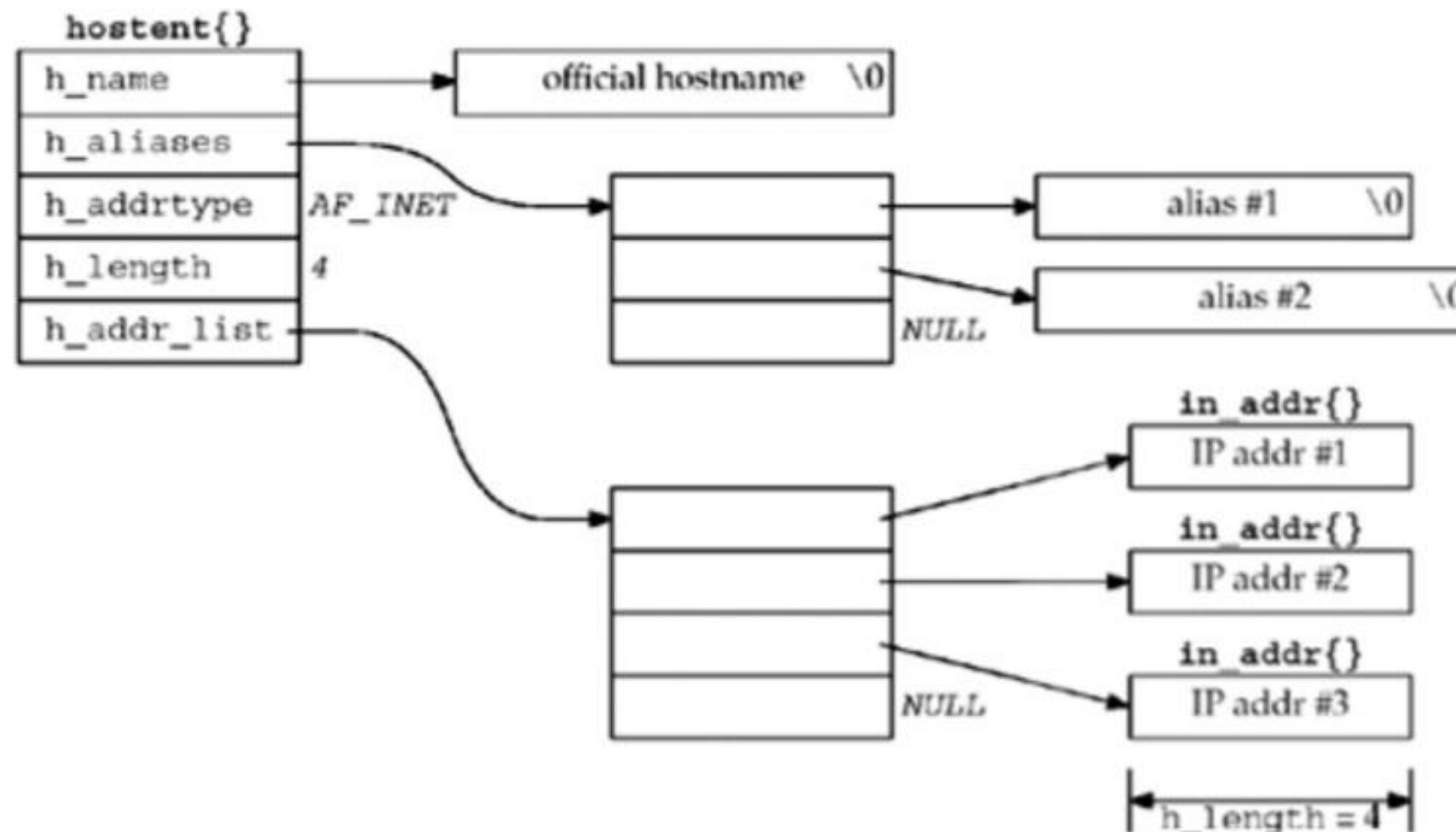
Geographic distribution

Different regional servers

Gethostbyname Function

- In terms of the DNS, gethostbyname performs a query for an A record. This function can return only IPv4 addresses.

Arrangements of hostent structure



- gethostbyname differs from the other socket functions as it does not set errno when an error occurs.
- Instead, it sets the global integer h_errno to one of the following constants defined by including <netdb.h>:
 - HOST_NOT_FOUND
 - TRY_AGAIN
 - NO_RECOVERY
 - NO_DATA (identical to NO_ADDRESS)

The NO_DATA error means the specified name is valid, but it does not have an A record.

Error Constant	Meaning (Simple)	Technical Meaning	When It Happens	Real Example
HOST_NOT_FOUND	Host doesn't exist	DNS server says domain is invalid	Wrong domain name	"www.abcxynotreal.com"
TRY_AGAIN	Temporary failure	DNS server not responding / timeout	Network issue or DNS busy	DNS server down
NO_RECOVERY	Non-recoverable error	Permanent DNS failure	Corrupt DNS response	Server misconfiguration
NO_DATA / NO_ADDRESSES	No IP for valid host	Domain exists but no A record	No IPv4 address available	Only IPv6 domain

gethostbyaddr Function

- The function `gethostbyaddr` takes a binary IPv4 address and tries to find the hostname corresponding to that address.
- This is the reverse of `gethostbyname`.

```
#include <netdb.h>
```

```
struct hostent *gethostbyaddr (const char *addr, socklen_t len, int family);
```

Returns: non-null pointer if OK, `NULL` on error with `h_errno` set

- This function returns a pointer to the same hostent structure
- The field of interest in this structure is normally h_name, the canonical hostname.
- The addr argument is not a char*, but is really a pointer to an in_addr structure containing the IPv4 address.
- len is the size of this structure: 4 for an IPv4 address.
- The family argument is AF_INET.
- In terms of the DNS, gethostbyaddr queries a name server for a PTR record in the in-addr.arpa domain

getservbyname and getservbyport Functions

- Services, like hosts, are often known by names, too.
- If a service is referred by its name in instead of by its port number, and if the mapping from the name to port number is contained in a file (normally /etc/services), then if the port number changes, all we need to modify is one line in the /etc/services file instead of having to recompile the applications.
- getservbyname looks up a service given its name.
- The canonical list of port numbers assigned to services is maintained by the IANA at <http://www.iana.org/assignments/port-numbers>.
- A given /etc/services file is likely to contain a subset of the IANA assignments.

http	80/tcp
https	443/tcp
ftp	21/tcp
smtp	25/tcp
domain	53/udp

getservbyname and getservbyport Functions

```
#include <netdb.h>
```

```
struct servent *getservbyname (const char *servname, const char *protoname);
```

Returns: non-null pointer if OK, `NULL` on error

This function returns a pointer to the following structure:

```
struct servent {  
    char    *s_name;        /* official service name */  
    char    **s_aliases;    /* alias list */  
    int      s_port;        /* port number, network-byte order */  
    char    *s_proto;       /* protocol to use */  
};
```

getservbyname and getservbyport Functions

- The service name servname must be specified.
- If a protocol is also specified (protoname is a non-null pointer), then the entry must also have a matching protocol.
- Some Internet services are provided using either TCP or UDP, while others support only a single protocol.
- If protoname is not specified and the service supports multiple protocols, it is implementation-dependent as to which port number is returned.
- Normally this does not matter, because services that support multiple protocols often use the same TCP and UDP port number, but this is not guaranteed

getservbyname and getservbyport Functions

- The main field of interest in the servent structure is the port number.
- Since the port number is returned in network byte order, we must not call htons when storing this into a socket address structure



Getaddrinfo Function

- The gethostbyname and gethostbyaddr functions only support IPv4.
- The function handles both name-to-address and service-to-port translation.
- It returns sockaddr structures instead of a list of addresses.
- These sockaddr structures can then be used by the socket functions directly.
- The application deals only with the socket address structures that are filled in by getaddrinfo.

```
#include <netdb.h>
```

```
int getaddrinfo (const char *hostname, const char *service, const struct  
addrinfo *hints, struct addrinfo **result) ;
```

Returns: 0 if OK, nonzero on error

Translation
Type

Example

Hostname
→ IP
address

google.com →
142.x.x.x

Service
name →
Port
number

http → 80

getaddrinfo()



returns addrinfo linked list



contains sockaddr structures



directly passed to:

socket()

connect()

bind()

getaddrinfo()



returns fully prepared sockaddr
structures



directly usable by socket(),
connect(), bind()

- This function returns through the result pointer a pointer to a linked list of addrinfo structures, which is defined by including <netdb.h>.

```

struct addrinfo {
    int      ai_flags;          /* AI_PASSIVE, AI_CANONNAME */
    int      ai_family;        /* AF_xxx */
    int      ai_socktype;      /* SOCK_xxx */
    int      ai_protocol;      /* 0 or IPPROTO_xxx for IPv4 and IPv6 */
    socklen_t ai_addrlen;      /* length of ai_addr */
    char     *ai_canonname;    /* ptr to canonical name for host */
    struct sockaddr *ai_addr;  /* ptr to socket address structure */
    struct addrinfo *ai_next;  /* ptr to next structure in linked list */
};
    
```

Feature	Older Functions	getaddrinfo ()
IPv4 support	✓	✓
IPv6 support	✗	✓
Protocol independent	✗	✓
Returns ready sockaddr	✗	✓
Thread safe	✗	✓
Handles service names	Partial	✓

Field	Technical Importance	Example
ai_flags	Controls how <code>getaddrinfo()</code> behaves during address generation and lookup	<code>AI_PASSIVE</code> , <code>AI_CANONNAME</code>
ai_family	Specifies whether the address should use IPv4 or IPv6	<code>AF_INET (IPv4)</code> , <code>AF_INET6 (IPv6)</code>
ai_socktype	Defines the transport communication model used by the socket	<code>SOCK_STREAM (TCP)</code> , <code>SOCK_DGRAM (UDP)</code>
ai_protocol	Identifies the exact protocol associated with the socket	<code>IPPROTO_TCP</code> , <code>IPPROTO_UDP</code>
ai_addrlen	Needed by socket functions to correctly read the address structure memory	<code>sizeof(sockaddr_in)</code>
ai_canonname	Stores the primary DNS hostname after resolution	<code>"google.com"</code>
ai_addr	Contains the real IP address and port number used for connection or binding	Pointer to <code>sockaddr_in</code> or <code>sockaddr_in6</code>
ai_next	Links multiple address results together as a linked list	Next IPv4/IPv6 address entry

- The hostname is either a hostname or an address string.
- The service is either a service name or a decimal port number string.
- hints is either a null pointer or a pointer to an addrinfo structure that the caller fills in with hints about the types of information the caller wants returned.
- For example, if the specified service is provided for both TCP and UDP (e.g., the domain service, which refers to a DNS server), the caller can set the ai_socktype member of the hints structure to SOCK_DGRAM.
- The only information returned will be for datagram sockets.

- If the function returns success (0), the variable pointed to by the result argument is filled in with a pointer to a linked list of addrinfo structures, linked through the ai_next pointer.
- There are two ways that multiple structures can be returned:
 1. If there are multiple addresses associated with the hostname, one structure is returned for each address that is usable with the requested address family (the ai_family hint, if specified).
 2. If the service is provided for multiple socket types, one structure can be returned for each socket type, depending on the ai_socktype hint. (Note that most getaddrinfo implementations consider a port number string to be implemented only by the socket type requested in ai_socktype; if ai_socktype is not specified, an error is returned instead.)

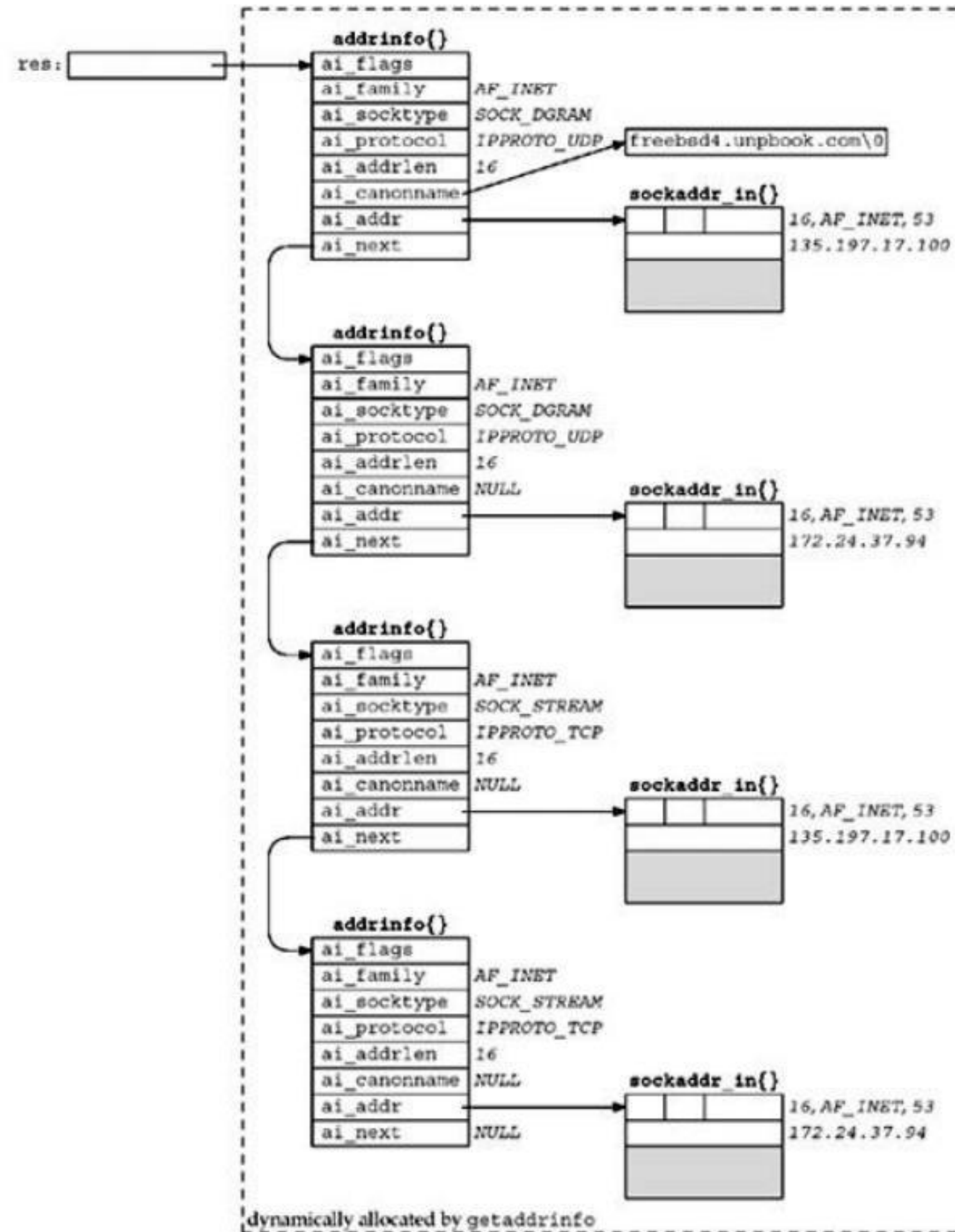
- The members of the hints structure that can be set by the caller are:

- `ai_flags` (zero or more `AI_XXX` values OR'ed together)
- `ai_family` (an `AF_XXX` value)
- `ai_socktype` (a `SOCK_XXX` value)
- `ai_protocol`

The possible values for the `ai_flags` member and their meanings are:

<code>AI_PASSIVE</code>	The caller will use the socket for a passive open.
<code>AI_CANONNAME</code>	Tells the function to return the canonical name of the host.
<code>AI_NUMERICHOST</code>	Prevents any kind of name-to-address mapping; the <i>hostname</i> argument must be an address string.
<code>AI_NUMERICSERV</code>	Prevents any kind of name-to-service mapping; the <i>service</i> argument must be a decimal port number string.
<code>AI_V4MAPPED</code>	If specified along with an <code>ai_family</code> of <code>AF_INET6</code> , then returns IPv4-mapped IPv6 addresses corresponding to A records if there are no available AAAA records.
<code>AI_ALL</code>	If specified along with <code>AI_V4MAPPED</code> , then returns IPv4-mapped IPv6 addresses in addition to any AAAA records belonging to the name.
<code>AI_ADDRCONFIG</code>	Only looks up addresses for a given IP version if there is one or more interface that is not a loopback interface configured with an IP address of that version.

- For example, if no hints are provided and if the domain service is looked up for a host with
- two IP addresses, four addrinfo structures are returned:
 - One for the first IP address and a socket type of SOCK_STREAM
 - One for the first IP address and a socket type of SOCK_DGRAM
 - One for the second IP address and a socket type of SOCK_STREAM
 - One for the second IP address and a socket type of SOCK_DGRAM



- Not guaranteed that an implementation should return the IP addresses in the same order as they are returned by the DNS.
- Some resolvers allow the administrator to specify an address sorting order in the `/etc/resolv.conf` file- which could affect the order of addresses returned by `getaddrinfo`.
- The information returned in the `addrinfo` structures is ready for a call to `socket` and then either a call to `connect` or `sendto` (for a client), or `bind` (for a server).
- The arguments to `socket` are the members `ai_family`, `ai_socktype`, and `ai_protocol`.
- The second and third arguments to either `connect` or `bind` are `ai_addr` (a pointer to a socket address structure of the appropriate type, filled in by `getaddrinfo`) and `ai_addrlen` (the length of this socket address structure).

- If we were to enumerate all 64 possible inputs to getaddrinfo (there are six input variables), many would be invalid and some would make little sense.
1. Specify the hostname and service. This is normal for a TCP or UDP client. On return, a TCP client loops through all returned IP addresses, calling socket and connect for each one, until the connection succeeds or until all addresses have been tried.
 2. For a UDP client, the socket address structure filled in by getaddrinfo would be used in a call to sendto or connect. If the client can tell that the first address doesn't appear to work (either by receiving an error on a connected UDP socket or by experiencing a timeout on an unconnected socket), additional addresses can be tried.
 3. If the client knows it handles only one type of socket (e.g., Telnet and FTP clients handle only TCP; TFTP clients handle only UDP), then the ai_socktype member of the hints structure should be specified as either SOCK_STREAM or SOCK_DGRAM.

4. A typical server specifies the service but not the hostname, and specifies the `AI_PASSIVE` flag in the hints structure. The socket address structures returned should contain an IP address of `INADDR_ANY` (for IPv4) or `IN6ADDR_ANY_INIT` (for IPv6). A TCP server then calls `socket`, `bind`, and `listen`. If the server wants to malloc another socket address structure to obtain the client's address from `accept`, the returned `ai_addrlen` value specifies this size.
5. A UDP server would call `socket`, `bind`, and then `recvfrom`. If the server wants to malloc another socket address structure to obtain the client's address from `recvfrom`, the returned `ai_addrlen` value specifies this size.
6. As with the typical client code, if the server knows it only handles one type of socket, the `ai_socktype` member of the hints structure should be set to either `SOCK_STREAM` or `SOCK_DGRAM`. This avoids having multiple structures returned, possibly with the wrong `ai_socktype` value.

7. The TCP servers that we have shown so far create one listening socket, and the UDP servers create one datagram socket. That is what we assume in the previous item. An alternate server design is for the server to handle multiple sockets using select or poll. In this scenario, the server would go through the entire list of structures returned by getaddrinfo, create one socket per structure, and use select or poll.
- The problem with this technique is that one reason for getaddrinfo returning multiple structures is when a service can be handled by IPv4 and IPv6.

- getaddrinfo is "better" than the gethostbyname and getservbyname functions
 - It makes it easier to write protocol-independent code;
 - One function handles both the hostname and the service and all the returned information is dynamically allocated, not statically allocated.

Disadvantages:

- Allocate a hints structure, initialize it to 0, fill in the desired fields, call getaddrinfo, and then traverse a linked list trying each one.

Number of addrinfo structures returned per IP address.

ai_socktype hint	Service is a name, service provided by:						Service is a port number
	TCP only	UDP only	SCTP only	TCP and UDP	TCP and SCTP	TCP, UDP, and SCTP	
0	1	1	1	2	2	3	error
SOCK_STREAM	1	error	1	1	2	2	2
SOCK_DGRAM	error	1	error	1	error	1	1
SOCK_SEQPACKET	error	error	1	error	1	1	1

gai_strerror Function

- The nonzero error return values from getaddrinfo have the names and meanings shown:

Constant	Description
EAI_AGAIN	Temporary failure in name resolution
EAI_BADFLAGS	Invalid value for <code>ai_flags</code>
EAI_FAIL	Unrecoverable failure in name resolution
EAI_FAMILY	<code>ai_family</code> not supported
EAI_MEMORY	Memory allocation failure
EAI_NONAME	<i>hostname</i> or <i>service</i> not provided, or not known
EAI_OVERFLOW	User argument buffer overflowed (<code>getnameinfo()</code> only)
EAI_SERVICE	<i>service</i> not supported for <code>ai_socktype</code>
EAI_SOCKTYPE	<code>ai_socktype</code> not supported
EAI_SYSTEM	System error returned in <code>errno</code>

```
#include <netdb.h>
```

```
const char *gai_strerror (int error);
```

Returns: pointer to string describing error message

freeaddrinfo Function

- All the storage returned by getaddrinfo, the addrinfo structures, the ai_addr structures, and the ai_canonname string are obtained dynamically (e.g., from malloc).
- This storage is returned by calling freeaddrinfo.

```
#include <netdb.h>
```

```
void freeaddrinfo (struct addrinfo *ai);
```

- ai should point to the first addrinfo structure returned by getaddrinfo.
- All the structures in the linked list are freed, along with any dynamic storage pointed to by those structures

Note:

1. Making a copy of just the addrinfo structure and not the structures that it in turn points to is called a shallow copy.
2. Copying the addrinfo structure and all the structures that it points to is called a deep copy.

getaddrinfo Function: IPv6

- Summary of how `getaddrinfo` handles IPv4 and IPv6 addresses. .

Hostname specified by caller	Address family specified by caller	Hostname string contains	Result	Action
non-null hostname string; active or passive	AF_UNSPEC	hostname	All AAAA records returned as <code>sockaddr_in6{ }s</code> and all A records returned as <code>sockaddr_in{ }s</code>	AAAA record search and A record search
		hex string	One <code>sockaddr_in6{ }</code>	<code>inet_pton(AF_INET6)</code>
		dotted-decimal	One <code>sockaddr_in{ }</code>	<code>inet_pton(AF_INET)</code>
	AF_INET6	hostname	All AAAA records returned as <code>sockaddr_in6{ }s</code>	AAAA record search
			If <code>ai_flags</code> contains <code>AI_V4MAPPED</code> , all AAAA records returned as <code>sockaddr_in6{ }s</code> else all A records returned as IPv4-mapped IPv6 <code>sockaddr_in6{ }s</code>	AAAA record search if no results then A record search
			If <code>ai_flags</code> contains <code>AI_V4MAPPED</code> and <code>AI_ALL</code> , all AAAA records returned as <code>sockaddr_in6{ }s</code> and all A records returned as IPv4-mapped IPv6 <code>sockaddr_in6{ }s</code>	AAAA record search and A record search
		hex string	One <code>sockaddr_in6{ }</code>	<code>inet_pton(AF_INET6)</code>
		dotted-decimal	Looked up as hostname	
	AF_INET	hostname	All A records returned as <code>sockaddr_in{ }s</code>	A record search
		hex string	Looked up as hostname	
		dotted-decimal	One <code>sockaddr_in{ }</code>	<code>inet_pton(AF_INET)</code>

- The "Result" column is what is to be returned to the caller, given the variables in the first three columns.
- The "Action" column is how the function obtains this result.

getaddrinfo Function: IPv6

null hostname string; passive	AF_UNSPEC	implied 0::0 implied 0.0.0.0	One sockaddr_in6{ } and one sockaddr_in{ }	inet_pton(AF_INET6) inet_pton(AF_INET)
	AF_INET6	implied 0::0	One sockaddr_in6{ }	inet_pton(AF_INET6)
	AF_INET	implied 0.0.0.0	One sockaddr_in{ }	inet_pton(AF_INET)
null hostname string; active	AF_UNSPEC	implied 0::1 implied 127.0.0.1	One sockaddr_in6{ } and one sockaddr_in{ }	inet_pton(AF_INET6) inet_pton(AF_INET)
	AF_INET6	implied 0::1	One sockaddr_in6{ }	inet_pton(AF_INET6)
	AF_INET	implied 127.0.0.1	One sockaddr_in{ }	inet_pton(AF_INET)

Remember:

- `getaddrinfo` deals with two different inputs
 1. The type of socket address structure the caller wants back.
 2. The type of records that should be searched for in the DNS or other database.
- The address family in the hints structure provided by the caller specifies the type of socket address structure that the caller expects to be returned.
- specifying `AF_UNSPEC` will return addresses that can be used with any protocol family that can be used with the hostname and service name.
- This implies that if a host has both AAAA records and A records, the AAAA records are returned as `sockaddr_in6` structures and the A records are returned as `sockaddr_in` structures.

Remember:

- If the `AI_PASSIVE` flag is specified without a hostname, then the IPv6 wildcard address (`IN6ADDR_ANY_INIT` or `0::0`) should be returned as a `sockaddr_in6` structure, along with the IPv4 wildcard address (`INADDR_ANY` or `0.0.0.0`), which is returned as a `sockaddr_in` structure.
- The address family specified in the hint structure's `ai_family` member, along with the flags such as `AI_V4MAPPED` and `AI_ALL` specified in the `ai_flags` member, dictate the type of records that are searched for in the DNS (A and/or AAAA) and what type of addresses are returned (IPv4, IPv6, and/or IPv4-mapped IPv6).

Remember:

- The hostname can also be either an IPv6 hex string or an IPv4 dotted-decimal string. The validity of this string depends on the address family specified by the caller. (AF_INET for IPV4, AF_INET6 for IPV6 and AF_UNSPEC for both)

host_serv Function

The hostname can also be either an IPv6 hex string or an IPv4 dotted-decimal string. The validity of this string depends on the address family specified by the caller. (AF_INET for IPV4, AF_INET6 for IPV6 and AF_UNSPEC for both)